EEL 5764 Computer Architecture

Systolic Array Scaling Under Input Bandwidth Limits

Project proposal

Ozlem Polat | UFID 93101169

Thomas Martin | UFID 79741555

Keith Kamer | UFID 60196560

Instructor: Professor Sandip Ray

University of Florida | Fall 2026

Systolic Array Scaling Under Input Bandwidth Limits

Problem and research question

Matrix multiplication benefits from parallel multiply-and-accumulate (MAC) units and is a common accelerator workload [1]. A larger array needs more operands each cycle. It also needs enough work to keep its processing elements (PEs) busy. This project asks: for a fixed matrix workload, when does a larger PE grid stop delivering proportional performance gains as the input-supply rate changes? We will examine the tradeoff between grid size, useful throughput, and FPGA resource use.

Our hypothesis is that larger grids will reduce cycle count when inputs are available at full rate. With eight 16-bit input words per cycle, the 8×8 grid should lose more of its potential gain than the 2×2 and 4×4 grids; sixteen words should remove this particular input limit. Short matrices may also leave a larger share of PEs idle during pipeline fill and drain. These are predictions to test; no speedup is assumed.

Proposed architecture

We will implement one output-stationary, two-dimensional systolic array in VHDL. Each PE receives an A operand from the left and a B operand from above, multiplies them when both are valid, and accumulates its assigned output. Boundary inputs are staggered so matching operands arrive together. A grid-size generic will generate 2×2, 4×4, and 8×8 arrays from the same PE design. All configurations will use signed 16-bit operands and an accumulator wide enough for an exact 16-term dot product.

The array will compute square matrix products of order 8 and 16. When a matrix is larger than the PE grid, the controller will divide its output into PE-sized tiles. It will stream the full inner dimension for each tile, store the completed tile, and proceed to the next. Thus, every configuration computes the same complete result. Results will be read out serially; we will report that cost separately from computation. The design will not include a processor interface, external DRAM controller, or alternate dataflow.

Experimental method

The 2×2 array is the architectural baseline. For each grid size, we will run both matrix sizes with a shared input budget of 4, 8, or 16 combined A/B words per clock. A P×P grid needs up to 2P boundary words to advance at full rate. Thus, the 2×2 grid can be fully supplied at four words, the 4×4 grid at eight, and the 8×8 grid at sixteen. RTL input staging will collect a complete boundary step before the grid advances. A stall will hold array state without losing operands.

We will compile and test the VHDL in Siemens QuestaSim, following the course tool guidance [11]. RTL simulation will verify operand timing and correct stalls; Vivado will estimate the cost and timing of the same PE array. Directed tests will include zero, identity, signed boundary, and mixed-sign matrices. Fixed-seed random cases will be checked against an independent software matrix multiplication. Tests will cover every array size, each workload, tile boundaries, and input stalls. We will record the tool version, matrix inputs, and simulation settings so the results can be repeated.

Synthesis setup and evaluation metrics

We will synthesize the three arrays in AMD Vivado for the same KV260 FPGA target, using identical synthesis settings and a 10 ns clock constraint [11]. The KV260 is a synthesis target; programming a board is not required. Questa cycle counts and Vivado reports answer different questions. We will report both, and use elapsed-time comparisons only for designs that meet the stated timing constraint. The input-budget sweep is a controlled interface model, not a measurement of KV260 DRAM bandwidth.

- Latency: compute cycles from the first accepted input to the last completed tile, including tile changes and stalls. We will also report total cycles through serial readout. For each workload and input budget, compute speedup is the 2×2 compute latency divided by the candidate latency.
- Useful throughput: completed MACs per measured cycle. Each order-N matrix product has N cubed useful MACs. We will report results per second at the constrained clock only for designs that meet timing.
- PE utilization and scaling: valid, useful MAC events divided by P squared times compute cycles for a P×P array. Scaling efficiency divides compute speedup by the PE-count increase relative to the 2×2 baseline.
- Hardware cost and timing: LUTs, flip-flops, DSP blocks, and block RAM from Vivado; timing slack at 10 ns after implementation where feasible. We will also compare useful throughput per DSP. If a design misses timing or exceeds resources, that outcome will be reported directly.

We will plot latency and utilization against grid size for each workload and input budget. A second plot will pair throughput with DSP use. Comparing the three input budgets will show whether a larger array is limited mainly by input delivery, finite workload size, or implementation cost. We will not infer power or energy savings from resource counts alone.

Relation to prior work

Prior studies already examine bandwidth, array shape, tiling, and memory stalls [2–4,8]. Other designs study FPGA resource mapping, new dataflows, or different algorithms [5–7,9]. Hardware validation of cycle models is also an active topic [10]. Our study applies a controlled input-rate comparison to one small, complete VHDL implementation.

Scope and expected result

The minimum complete study is a verified parameterized array; correct 8×8 and 16×16 matrix products on the 2×2, 4×4, and 8×8 grids; the three input budgets; and comparable simulation and synthesis results. Work will proceed in that order: verify one PE and a 2×2 tile, add tiling and parameterization, introduce stalls, then run the sweep and synthesize each configuration. Functional correctness is the gate before performance measurements.

The study may find a bandwidth knee, or it may show that fill, drain, and tile overhead dominate these small workloads. Either outcome answers the stated question if the workloads and constraints are held constant. A 32×32 workload, a second dataflow, precision changes, physical FPGA execution, and a CPU comparison are outside the committed scope. The final deliverables are VHDL source, a self-checking testbench, repeatable run settings, measured plots, and an architectural explanation of the results.

References

[1] N. P. Jouppi et al., “In-Datacenter Performance Analysis of a Tensor Processing Unit,” ISCA, 2017. doi:10.1145/3079856.3080246.

[2] A. Samajdar et al., “SCALE-Sim: Systolic CNN Accelerator Simulator,” 2019. arXiv:1811.02883.

[3] H. Genc et al., “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration,” DAC, 2021. arXiv:1911.09925.

[4] A. C. Yüzügüler et al., “Scale-out Systolic Arrays,” 2022. arXiv:2203.11540.

[5] J. Li et al., “Revealing Untapped DSP Optimization Potentials for FPGA-Based Systolic Matrix Engines,” FPL, 2024. arXiv:2409.03508.

[6] A. J. Abdelmaksoud et al., “DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration,” IEEE TCAS-I, 2025. doi:10.1109/TCSI.2025.3591960.

[7] R. Li and S. Chen, “Design and Implementation of an FPGA-Based Hardware Accelerator for Transformer,” 2025. arXiv:2503.16731.

[8] R. Raj et al., “SCALE-Sim v3: A Modular Cycle-Accurate Systolic Accelerator Simulator for End-to-End System Analysis,” 2025. arXiv:2504.15377.

[9] T. E. Pogue and N. Nicolici, “Strassen Multisystolic Array Hardware Architectures,” IEEE TVLSI, 2025. doi:10.1109/TVLSI.2025.3530785.

[10] J. Dang et al., “SCALE-Sim TPU: Validating and Extending SCALE-Sim for TPUs,” MLBench/ASPLOS Workshop, 2026. arXiv:2603.22535.

[11] UF EEL5721, “Reconfigurable Computing” course documentation, https://eel5721-reconfigurable-computing.github.io/documentation/